



Lab 03: Interact with Terraform Modules

Build Infrastructure with Terraform | GSP751

Overview

Used modules from the Terraform Registry to provision a VPC network, then built a custom module to manage a Cloud Storage static website bucket.

What are Terraform Modules?

Benefit	Description
Organize	Keep related configuration together for easier navigation
Encapsulate	Prevent unintended consequences between parts of configuration
Re-use	Share configuration across projects and teams
Consistency	Ensure best practices are applied consistently across all configurations

Task 1 - Use a Module from the Registry

Cloned the terraform-google-network repository and used the Network module to create a VPC with 3 subnets.

```
git clone https://github.com/terraform-google-modules/terraform-google-network
cd terraform-google-network
git checkout tags/v6.0.1 -b v6.0.1
```

Module configuration in main.tf:

```
module "test-vpc-module" {
  source      = "terraform-google-modules/network/google"
  version     = "~> 6.0"
  project_id  = var.project_id
  network_name = var.network_name
  mtu         = 1460
  subnets = [
    { subnet_name = "subnet-01" subnet_ip = "10.10.10.0/24" subnet_region =
"[REGION]" },
    { subnet_name = "subnet-02" subnet_ip = "10.10.20.0/24" subnet_region =
"[REGION]" },
    { subnet_name = "subnet-03" subnet_ip = "10.10.30.0/24" subnet_region =
"[REGION]" }
  ]
}
```

Used Gemini Code Assist to update variables.tf with project_id default and define network_name variable.

```
terraform init
terraform apply
```

Outputs:

```
network_name = "example-vpc"
network_self_link = "https://www.googleapis.com/compute/v1/projects/qwiklabs-gcp-02-804d8bdb1b17/global/networks/example-vpc"
project_id = "qwiklabs-gcp-02-804d8bdb1b17"
route_names = []
subnets_flow_logs = [
  false,
  true,
  true,
]
subnets_ips = [
  "10.10.10.0/24",
  "10.10.20.0/24",
  "10.10.30.0/24",
]
subnets_names = [
  "subnet-01",
  "subnet-02",
  "subnet-03",
]
subnets_private_access = [
  false,
  true,
  false,
]
subnets_regions = [
  "us-east1",
  "us-east1",
  "us-east1",
]
subnets_secondary_ranges = [
  tolist([]),
  tolist([]),
  tolist([]),
]
student_01_09a9adbfa267@cloudshell:~/terraform-google-network/examples/simple_project (qwiklabs-gcp-02-804d8bdb1b17) $
```

Task 2 - Build a Custom Module

Created a gcs-static-website-bucket module to manage Cloud Storage buckets for static website hosting.

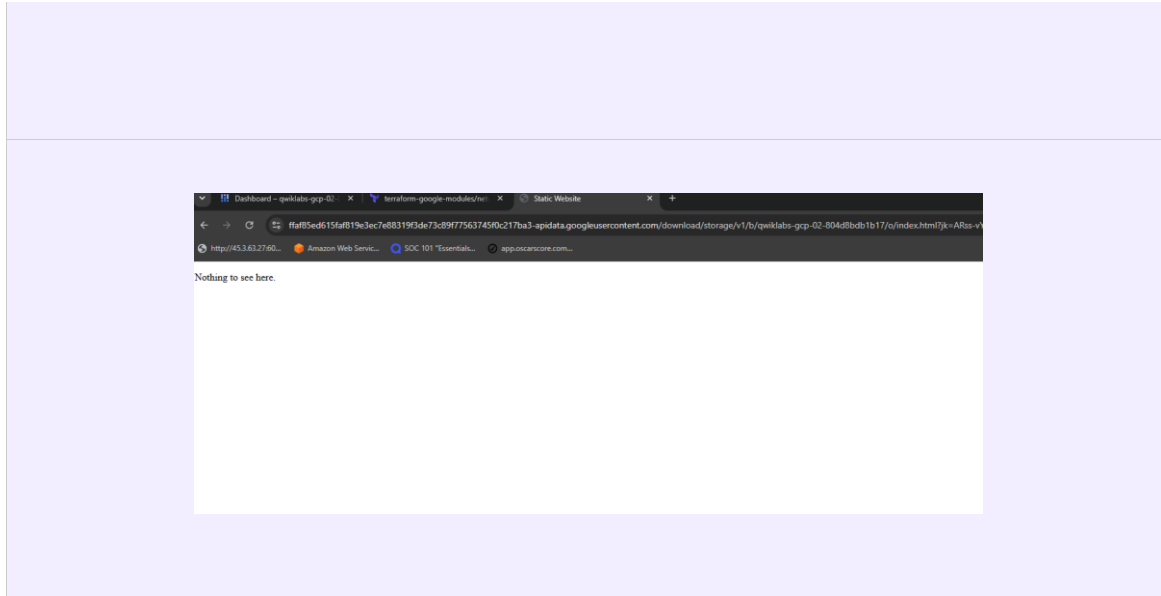
Module structure:

```
main.tf
modules/
  gcs-static-website-bucket/
    website.tf
    variables.tf
    outputs.tf
    README.md
    LICENSE
```

Root main.tf referencing the custom module:

```
module "gcs-static-website-bucket" {
  source      = "./modules/gcs-static-website-bucket"
  name        = var.name
  project_id  = var.project_id
  location    = "[REGION]"
  lifecycle_rules = [{ action = { type = "Delete" } condition = { age = 365 } }]
}
```

```
terraform init
terraform apply
gsutil cp *.html gs://[PROJECT_ID]
```



What I Learned

- Modules are reusable sets of Terraform configuration files in a single directory
- The Terraform Registry provides community and official modules for common infrastructure
- Module input variables are set as arguments in the module block
- Module outputs are accessed as `module.MODULE_NAME.OUTPUT_NAME`
- `terraform init` or `terraform get` must be run to install modules before use
- Custom modules follow the same structure as root modules with `main.tf`, `variables.tf`, `outputs.tf`

Resources

Terraform Registry: <https://registry.terraform.io> | Lab ID: GSP751